Data Structures (ETCCDS202)

Unit–4 Assignment: Non-Linear & Advanced Data Structures

Course Details

Name of the School: School of Engineering & Technology

Program (Class / Semester / Batch): B.Tech (CSE /Robotics and AI) Sem-A

Course Title: Data Structures

Course Code: ETCCDS202

Unit Number: 4

Unit Title: Sorting Algorithms

Student Name: Ajitesh Singh Gusain

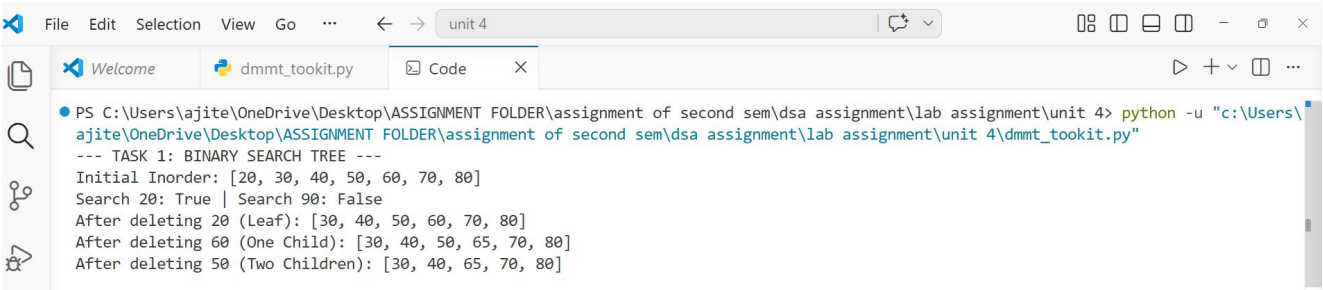
Roll No. :2501630014

Submitted to: Dr . Aarti Sangwan

Task 1: Binary Search Tree (BST) Operations Goal: Build an ordered structure that supports efficient search and sorted traversal.

1. Binary Search Tree (BST) Operations

- Sorted In order Traversal: In a Binary Search Tree, the left subtree contains only nodes with keys lesser than the root, and the right subtree contains only nodes with keys greater than the root. By following the In order pattern (Left > Root > Right), we naturally visit these elements in their strictly increasing numerical order.
- Time Complexity Analysis:
    - Average Case O(\log n): In a balanced tree, each comparison allows us to discard approximately half of the remaining search space, leading to logarithmic time performance.
    - Worst Case O(n): This occurs if the tree becomes "skewed" (resembling a linked list), requiring the algorithm to traverse every single node.



Task 2: Graph Representation + BFS and DFS

BFS vs. DFS: Breadth-First Search (BFS) explores the graph level-by-level, visiting all immediate neighbours before moving to the next depth. In contrast, Depth-First Search (DFS) explores as far as possible along a single branch before backtracking to explore other paths.

Real-World Applications:

- BFS: Widely used for finding the shortest path in unweighted networks, such as GPS navigation systems.
- DFS: Frequently utilized for cycle detection in circuits or topological sorting for complex task scheduling.

```
--- TASK 2: GRAPH TRAVERSAL ---
Adjacency List: {'A': [('B', 2), ('C', 4)], 'B': [('D', 7), ('E', 3)], 'C': [('E', 1), ('F', --- TASK 2: GRAPH TRAVERSAL ---
Adjacency List: {'A': [('B', 2), ('C', 4)], 'B': [('D', 7), ('E', 3)], 'C': [('E', 1), ('F', 8)], 'D': [('F', 5)], 'E': [('D', 2), ('F',
 6)]}
BFS from A: ['A', 'B', 'C', 'D', 'E', 'F']
DFS from A: ['A', 'B', 'D', 'F', 'E', 'C']
```

Task 3: Hash Table with Separate Chaining (Collision Handling)

BFS vs. DFS: Breadth-First Search (BFS) explores the graph level-by-level, visiting all immediate neighbours before moving to the next depth. In contrast, Depth-First Search (DFS) explores as far as possible along a single branch before backtracking to explore other paths.

Real-World Applications:

- BFS: Widely used for finding the shortest path in unweighted networks, such as GPS navigation systems.
- DFS: Frequently utilized for cycle detection in circuits or topological sorting for complex task scheduling.

```
--- TASK 3: HASH TABLE (SIZE 5) ---
Hash Table Structure: [[(10, 'Value_10'), (15, 'Value_15'), (20, 'Value_20')], [], [(7, 'Value_7'), (12, 'Value_12')], [], []]
Get key 7: Value_7
Bucket 0 after deleting 15: [(10, 'Value_10'), (20, 'Value_20')]
PS C:\Users\ajite\OneDrive\Desktop\ASSIGNMENT FOLDER\assignment of second sem\dsa assignment\lab assignment\unit 4>
```

Dmmt_toolkit.py

```python
"""
Project: Data Management Mini Toolkit (DMMT)
Student: Ajitesh Singh Gusain
Course: B.Tech CSE (Robotics & AI) - Semester 2
University: K.R. Mangalam University
"""

class BSTNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, key):
        self.root = self._insert_recursive(self.root, key)

    def _insert_recursive(self, node, key):
        if not node:
            return BSTNode(key)
        if key < node.key:
            node.left = self._insert_recursive(node.left, key)
        else:
            node.right = self._insert_recursive(node.right, key)
        return node

    def search(self, key):
        curr = self.root
        while curr:
            if curr.key == key: return True
            curr = curr.left if key < curr.key else curr.right
        return False

    def inorder_traversal(self):
        res = []
        self._inorder_helper(self.root, res)
        return res

    def _inorder_helper(self, node, res):
        if node:
            self._inorder_helper(node.left, res)
            res.append(node.key)
            self._inorder_helper(node.right, res)

    def delete(self, key):
        self.root = self._delete_recursive(self.root, key)
```

```python
    def _delete_recursive(self, node, key):
        if not node: return None
        if key < node.key:
            node.left = self._delete_recursive(node.left, key)
        elif key > node.key:
            node.right = self._delete_recursive(node.right, key)
        else:
            # Case 1 & 2: No child or one child
            if not node.left: return node.right
            if not node.right: return node.left
            # Case 3: Two children
            temp = self._min_value_node(node.right)
            node.key = temp.key
            node.right = self._delete_recursive(node.right, temp.key)
        return node

    def _min_value_node(self, node):
        current = node
        while current.left:
            current = current.left
        return current

lass Graph:
    def __init__(self):
        self.adj = {}

    def add_edge(self, u, v, w):
        if u not in self.adj: self.adj[u] = []
        self.adj[u].append((v, w))

    def bfs(self, start):
        visited, queue, order = {start}, [start], []
        while queue:
            u = queue.pop(0)
            order.append(u)
            for v, w in self.adj.get(u, []):
                if v not in visited:
                    visited.add(v)
                    queue.append(v)
        return order

    def dfs(self, start):
        visited, order = set(), []
        def _visit(u):
            visited.add(u)
            order.append(u)
            for v, w in self.adj.get(u, []):
                if v not in visited: _visit(v)
        _visit(start)
        return order
```

```python
class HashTable:
    def __init__(self, size=5):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        return key % self.size

    def insert(self, key, value):
        idx = self._hash(key)
        for i, (k, v) in enumerate(self.table[idx]):
            if k == key:
                self.table[idx][i] = (key, value)
                return
        self.table[idx].append((key, value))

    def get(self, key):
        idx = self._hash(key)
        for k, v in self.table[idx]:
            if k == key: return v
        return None

    def delete(self, key):
        idx = self._hash(key)
        self.table[idx] = [p for p in self.table[idx] if p[0] != key]

def main():
    print("--- TASK 1: BINARY SEARCH TREE ---")
    bst = BST()
    for x in [50, 30, 70, 20, 40, 60, 80]: bst.insert(x)
    print(f"Initial Inorder: {bst.inorder_traversal()}")
    print(f"Search 20: {bst.search(20)} | Search 90: {bst.search(90)}")

    bst.delete(20) # Leaf node case
    print(f"After deleting 20 (Leaf): {bst.inorder_traversal()}")

    bst.insert(65)
    bst.delete(60) # One child case
    print(f"After deleting 60 (One Child): {bst.inorder_traversal()}")

    bst.delete(50) # Two children case
    print(f"After deleting 50 (Two Children): {bst.inorder_traversal()}")

    print("\n--- TASK 2: GRAPH TRAVERSAL ---")
    g = Graph()
    edges = [('A','B',2), ('A','C',4), ('B','D',7), ('B','E',3),
             ('C','E',1), ('D','F',5), ('E','D',2), ('E','F',6), ('C','F',8)]
    for u, v, w in edges: g.add_edge(u, v, w)

    print(f"Adjacency List: {g.adj}")
    print(f"BFS from A: {g.bfs('A')}")
    print(f"DFS from A: {g.dfs('A')}")
```

```python
    print("\n--- TASK 3: HASH TABLE (SIZE 5) ---")
    ht = HashTable(5)
    for k in [10, 15, 20, 7, 12]: ht.insert(k, f"Value_{k}")
    print(f"Hash Table Structure: {ht.table}")
    print(f"Get key 7: {ht.get(7)}")
    ht.delete(15)
    print(f"Bucket 0 after deleting 15: {ht.table[0]}")

if __name__ == "__main__":
    main()
```